Exceptions, Streams and File I/O

How do we catch an exception?

We can catch an exception in either of two ways. They are:

- By try-catch block
- By using throws clause

Java provides five essential keywords to handle an exception. They are:

- try
- catch
- finally
- throw
- throws

Why use Finally ?

```
try {
    a;
    block;
    off;
    statements;
} catch (Exception e) {
    handle;
    exception;
    e;
} finally {
    do;
    some;
    cleanup;
}
```

Versus

```
try {
    a;
    block;
    off;
    statements;
} catch (Exception e) {
    handle;
    exception;
    e;
}
do;
some;
cleanup;
```

Why use Finally ?

```
public class Main {

    public static void main(String[] args) throws Exception {
        try {
            System.out.println('A');
            int b = 99 / 1;
            throw new IOException();
        }
      catch (ArithmeticException ex) {
            System.out.println("D");
        }
        finally {
            System.out.println("F");
        }
        System.out.println("G");
    }
}
```

IO Exception thrown here is not handled

Finally block will be executed!

This will not be executed.

Output will be: A F and then compiler will throw IO Exception

So will finally ALWAYS be executed ?

```
public static void main(String[] args)
    try {
        System.out.println('A');
        int b = 99 / 0;
    }
    catch (ArithmeticException ex) {
        System.out.println("D");
        System.exit( status: 0);
    }
    finally {
        System.out.println("F");
    }
    System.out.println("G");
}
```

Arithmetic Exception

Program is terminated

This will not be executed.

This is the only scenario where finally block will not be executed.

Finally in Methods

```
public static int Demo(){
    try{
        System.out.println("Inside Try Block");
        int b=5/0;
        return 1;
    }
    catch (Exception e){
        System.out.println("Inside Catch Block");
        return 2;
    }
    finally{
        System.out.println("Inside Finally Block");
        return 3;
    }
}
public static void main(String[] args)  {
    int a = Demo();
    System.out.println("The value returned is: " + a);
}
```

Arithmetic Exception

Exception handled

Finally will be called

The value returned is: 3

Method overloading - Exceptions

- If super class method is not throwing any exceptions, then it can be overrided with any unchecked type of exceptions, but can not be overrided with checked type of exceptions.
- If a super class method is throwing unchecked exception, then it can be overrided in the sub class with same exception or any other unchecked exceptions but can not be overrided with checked exceptions.
- If super class method is throwing checked type of exception, then it can be overrided with same exception or with it's sub class exceptions i.e you can decrease the scope of the exception, but can not be overrided with it's super class exceptions i.e you can not increase the scope of the exception.

I/O Stream

- In general:
  - **Stream** - a sequence of data
  - **Input Stream** – reads data from source
  - **Output Stream** – writes data to destination

A **stream** is a method to sequentially access a file. **I/O stream** means an input source or output destination representing different types of sources. E.g. disk files, console, etc.

The `java.io` package provides classes that allow you to convert between Unicode character streams and byte streams of non-Unicode text.

Unicode character – 16 bits = 2 bytes
Byte – 8 bits = 1 byte

Output Stream

Programs need a way to output data to a *screen(console)*, *file*, or elsewhere. An **OutputStream** is a class that supports output.
OutputStream provides several methods for writing sequence of **bytes** to a destination.
The sequence is normally placed into a buffer, and the system then outputs the buffer at various times.

System.out -> It's the system's **standard output**
out is a variable in **System** class, which is derived from `OutputStream` called a `PrintStream`.

print() and println() ->

- Support **primitive types** (int, boolean, char, …)
  - → Convert to a **sequence of characters**
- Also support **String**s (note that String is NOT a primitive type)
  - Why?
  - → String is just a string of characters, in other words, it's a **sequence of characters**
- For **reference types** (array, ArrayList, …)
  - → Object's class name followed by "@", and the hexadecimal value of the object's hash code – which is the address in memory.

Input Stream

Program needs to receive input data, whether from a keyboard, touchscreen, or elsewhere. An **InputStream** is a class for achieving such input.

`read()` method – extracting bytes from a particular source.

System.in

- It's a predefined **InputStream** object, that means, we don't need to import the class before using it (same as System.out).
- It's associated with System's standard input.
- It reads the standard input from buffer, then store in your program.

```
Scanner myObj = new Scanner(System.in); // Create a Scanner object
```

Output formatting

A programmer can adjust the way that a program's output appears, a task known as **output formatting**.

The standard output stream `System.out` provides the method `printf()` for output formatting.

The first argument of the printf() method, the **format string**, specifies the format of the text to print along with any number of placeholders for printing numeric values.

The placeholders are known as **format specifiers**. A **format specifier** specifies the type of value to print in its place. A format specifier begins with the % character followed by another character that indicates the value type to be printed.

Ex: %d indicates an integer type, and %s indicates a string type.

Output formatting

```
String account = "Prime";
double total = 210.35;
int years = 5;


System.out.printf("The %s account saved you $%f over %d years\n",
account, total, years);
```

Output: The Prime account saved you $210.350000 over 5 years.

Explanation: account variable is substitute for %s, total variable is substitute for %f, and years variable is substitute for %d. Floating-point values (float, double) are output to 6 decimal places by default.

Output formatting

Format specifier	Data type(s)	Notes
%c	char	Prints a single Unicode character
%d	int, long, short	Prints a decimal integer value.
%o	int, long, short	Prints an octal integer value.
%h	int, char, long, short	Prints a hexadecimal integer value.
%f	float, double	Prints a floating-point value.
%e	float, double	Prints a floating-point value in scientific notation.
%s	String	Prints the characters in a String variable or literal.
%%		Prints the "%" character.
%n		Prints the platform-specific new-line character.

Output formatting – floating point numbers

```
%(flags)(width)(.precision)specifier
```

Width specifies the **minimum number of characters to print**.

- If the formatted value has **more characters than the width**, the value **will NOT** be truncated.
- If the formatted value has **fewer characters than the width**, the output **will be** padded with spaces (or 0's if the '0' flag is specified).

Precision specifies the number of digits to print following the decimal point. If the precision is not specified, a default precision of 6 is used.

Flags:

- +: Prints a preceding + sign for positive values. Negative numbers are always printed with the - sign.
- 0: Pads the output with 0's when the formatted value has fewer characters than the width.
- -: Left aligns the output given the specified width, padding the output with spaces.

Output formatting – floating point numbers

```
String name = "Alice";
double gpa = 3.5;
int age = 20;
```

```
System.out.printf("%d - %s", age,
name);
```
20 - Alice

```
System.out.printf(%f%% %d”, gpa,
age);
```
3.500000% 20

```
System.out.printf(“%d, %d”, age,
age+1);
```
20, 21

```
float myFloat = 12.34f;
printf("Value: %07.2f", myFloat);
```
Value:0012.34

Note that there are 2 spaces at front.

```
printf("%+f", myFloat);
```
+12.340000

```
printf("%08.2f", myFloat);
```
00012.34

Output formatting – Integers

```
%(flags)(width)specifier
```

Width specifies the **minimum number of characters to print**.

- If the formatted value has **more characters than the width**, the value **will NOT** be truncated.
- If the formatted value has **fewer characters than the width**, the output **will be** padded with spaces (or 0's if the '0' flag is specified).

Flags:

- +: Prints a preceding + sign for positive values. Negative numbers are always printed with the - sign.
- 0: Pads the output with 0's when the formatted value has fewer characters than the width.
- -: Left aligns the output given the specified width, padding the output with spaces.

Output formatting – Strings

```
%(flags)(width)(.precision)specifier
```

Width specifies the **minimum number of characters to print**.

- If the formatted value has **more characters than the width**, the value **will NOT** be truncated.
- If the formatted value has **fewer characters than the width**, the output **will be** padded with spaces.

Precision specifies the **maximum number of characters** to print. If the string has more characters than the precision, the string **will be truncated**.

Flags:

- -: Left aligns the output given the specified width, padding the output with spaces.

Flushing the output

- To preserve resources, the system may wait until the buffer is full, or at least has a certain number of characters before moving them to the output device.

- In general, when you write data to a stream, it is not written immediately, and it is buffered. So we use flush() when you need to be sure that all your data from buffer is written.

- The statement `System.out.flush();` writes the contents of the buffer for `System.out` to the computer screen. **Most Java implementations make `System.out` flush when a newline character is output or println() method is called. It also flushes automatically when the program ends normally.**

Flushing the output (Example)

Indicate if the line of output is immediately flushed from the System.out buffer

```
System.out.print("My life has been the poem ");  // A
System.out.println("I would have writ");          // B
System.out.print("But I could not ");             // C
System.out.flush();
System.out.print("both live and utter it.");  // D EOP
```

Line A: False, because print() is not outputting a newline character.
Line B: True, because println() outputs a newline character. It also flushes line A.
Line C: False, because print() is not outputting a newline character. When flush() is called on the next line, the output is flushed.
Line D: True, because it's at the end of program.

Streams using Strings

- Sometimes a programmer wishes to read input data from a string rather than from the keyboard (standard input).
- A programmer can associate a Scanner object with a String rather than with the keyboard (standard input).
- Such an object can be used just like a Scanner object associated with the System.in stream. A Scanner object initialized from a String is often referred to as an **input string stream**.

```
Scanner scanner = new Scanner(date); //scan a String
```

Streams using Strings

- Similarly, a new **output string stream** can be created that is associated with a String rather than with the screen.
- An output string stream is created using both the `StringWriter` and `PrintWriter` classes.

- import java.io.StringWriter;
- import java.io.PrintWriter;

File I/O

- File Byte Streams
  - `FileInputStream`
  - `FileOutputStream`

- File Character Streams
  - `FileReader, BufferedFileReader`
  - `FileWriter, BufferedFileWriter`

File I/O General Formal

1. Import the required header files

```
import java.util.Scanner;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.PrintWriter;
import java.io.IOException;
```

2. Open the file for reading / writing (or both) and create a input/output stream
3. Check for exceptions and handle them
4. Use scanner to read data from file
5. Use printwriter or stringwriter to write contents to the buffer
6. Use flush to ensure all contents are pushed from buffer to file
7. Close the files

File Byte Streams

To read file input, a programmer can create a new input stream that comes from a **file**.

```
FileInputStream inputFile = new
FileInputStream(“a/file/path.txt”);
```

You can use a Scanner to get the data inside this file.

```
Scanner scanner = new Scanner (inputFile);
```

A `FileOutputStream` is a class that supports writing to a file. `FileOutputStream` only contains methods for **writing bytes**. `FileOutputStream` will create output file if the file does not exist in the path.

```
FileOutputStream outputFile = new
FileOutputStream("helloWorld.txt");
```

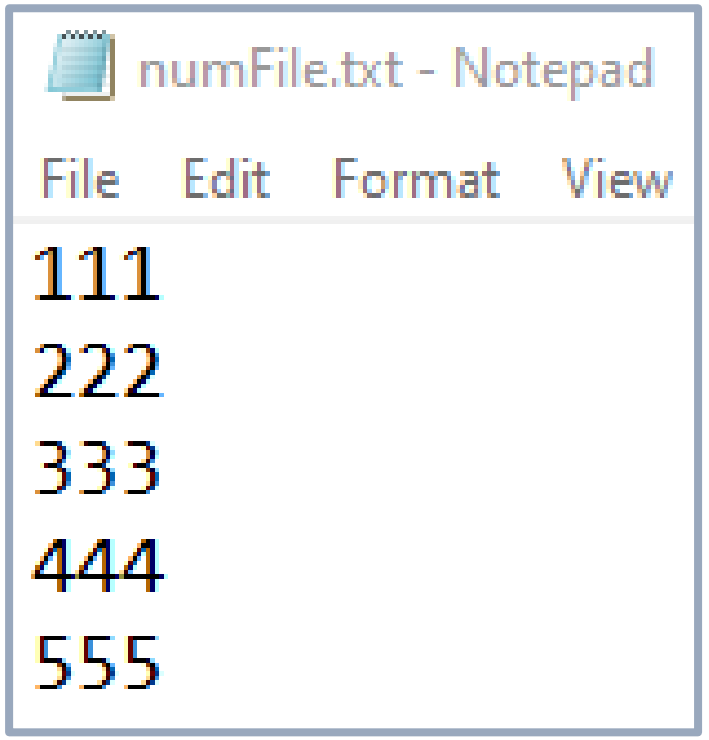
Create a PrintWriter to write to the file

```
PrintWriter writer = new PrintWriter(outputFile);
```

Example – Accessing data from a file

```
FileInputStream inputFile = new
FileInputStream("numFile.txt");
Scanner scanner = new Scanner (inputFile);
int fileNum;

while(scanner.hasNextInt()){
    fileNum = scanner.nextInt();
    System.out.println("num: " + fileNum);
}
inputFile.close();
```

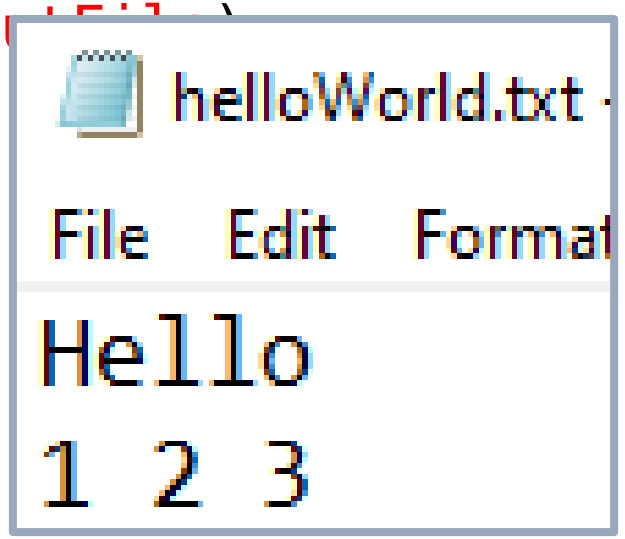


Reading until the end of the file – use hasNext() or hasNextInt() in Scanner.

Example – Writing data to a file

```
FileOutputStream outputFile = new
FileOutputStream("./helloWorld.txt");
PrintWriter writer = new PrintWriter(outputFile);

writer.println("Hello");
writer.println("1 2 3");
writer.flush();
outputFile.close();
```



To write strings and other data types, a `PrintWriter` is commonly used. A `PrintWriter` has methods such as `print()` and `println()`.

File Character Streams

It is recommended **not** to use the `FileInputStream` and `FileOutputStream` classes if you have to read and write any textual information as these are Byte Stream classes.

```
FileInputStream -> FileReader
FileOutputStream -> FileWriter
```

The `BufferedReader` and `BufferedWriter` classes use an internal buffer to store data while reading and writing, respectively, which is more efficient when dealing with characters.

```
Scanner -> BufferedReader
PrintWriter/StringWriter -> BufferedWriter
```

Close the FileReader, FileWriter, `BufferedReader` and `BufferedWriter` after user

```
bw.close(); fr.close(); br.close(); bw.close();
```

Paths

A **path** is a way to refer to a particular file or directory in a file system.

Absolute path

A path is *absolute* if it starts with the root element of the file system. In Windows, the root element is a drive e.g. “C:\\”, while in linux/unix is denoted by “/” character.

Relative path

A *relative path* is a path which doesn’t start with the root element of the file system. It is simply the path needed in order to locate file from within the current directory of your program.

It is a common convention to use a relative path when locating file inside your program.

Relative Paths

Let's say you have your *Program3.java* source file at

C:\CS160\CS160Assignments\Program3\Program3.java

And you have another file *Protein.txt* under the same directory:

C:\CS160\CS160Assignments\Program3\Protein.txt

If you want to access and create a byte stream for *Protein.txt* in *Program3.java*, you need to write

```
FileInputStream inputFile = new
FileInputStream("Protein.txt");
```

The path of Protein.txt is relative to Program3.java.

Relative Paths (IDEs – Eclipse, IntelliJ)

Let's say you have your *Program3.java* source file at

C:\CS108\CS108Assignments\Program3\src\Program3.java

And you have another file *Protein.txt* under the same directory:

C:\CS108\CS108Assignments\Program3\Protein.txt

If you want to access and create a byte stream for *Protein.txt* in *Program3.java*, you need to write

```
FileInputStream inputFile = new FileInputStream("Protein.txt");
```

Note: the file path in IntelliJ project is relative to the working directory (default is the root directory) of your project! Not your .java file!

In-class Activity

Write a JAVA Program that would:

1. Read integers from a text file – input.txt
2. Increment the values by 1 and write it to a new text file – output.txt
3. Your code should handle the exceptions

HINT: Program structure

1. Load required headers
2. use try-catch block to handle the IOException
3. Inside the try block – open the files – read data - do computation – write data – close files
4. catch block would just print an error message